हेलो! आपका स्वागत है **Unit 10: Advanced Functions & Scope** की complete study class में.

मैंने आपके workspace में, unit 10 adv functions folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter10_advanced_functions.md
- Code Examples File: advanced_functions_examples.js
- Practice Exercises File: advanced_functions_practice.js

चलिए अब Advanced Functions & Scope को एकदम clear and simple Hinglish explanation के साथ समझना शुरू करते हैं.

PART 1: SCOPES & SCOPE CHAIN MECHANISMS

JavaScript variables accessibility scope properties ke through define hoti hai.

1. Lexical Scope

What is it? Lexical Scope (Static Scope) ka matlab hai ki kisi variable ka access constraints boundary uski code file mein physical position (where it is written) se decide hota hai. Inner nested functions apne surrounding parent block variables ko read/access kar sakte hain.

Why is it important? Compiler execution phase start hone se pehle hi structural coordinates mapping complete kar leta hai, jisse variables scopes values secure rehti hain.

Simple example

javascript

```
let globalVal = "I am Global";

function outer() {
  let outerVal = "I am Outer";
  function inner() {
    console.log(outerVal); // inner can access outer
    console.log(globalVal); // inner can access global
  }
  inner();
}
outer();
```

Output I am Outer I am Global

---

2. Scope Chain

What is it? Scope Chain variables search lookup mechanism hai.

How does it work? Jab compiler ko local scope variable nahi milta, toh woh parent boundaries lookup checks repeat karta hai, jab tak global object values boundary tak match checks success nahi ho jata. Nahi milne par program ReferenceError show karta hai.

Simple example

javascript

```
let a = 10;

function parent() {
   let b = 20;
   function child() {
      let c = 30;
      console.log(a + b + c); // b and a resolved using Scope Chain
   }
   child();
}

parent();
```

Output 60

---

3. Execution Context & Call Stack

What is it?

- **Execution Context:** Yeh ek dynamic workspace container hota hai jisme code compile aur line-by-line execute hota hai.
- **Call Stack:** Ek order tracker (LIFO - Last In First Out stack) jo run instances tracks hold karta hai.

How does it work?

1. Execution Context runs in 2 phases:

- Memory allocation phase: registers variables name (sets var as undefined) and functions.
- Execution phase: variables assignment coordinates values run.

2. Call stack tracks which execution context is currently active. When a function finishes execution, its context is popped off the stack.

PART 2: HOISTING & TEMPORAL DEAD ZONE

1. Hoisting

What is it? Compilation setup registers code variables functions allocation structures lines memory slots pehle allocation parameters check handles.

How does it work?

- **var:** values hoisted setting default initialized to undefined.
- **let & const:** Hoisted blocks allocations sets to uninitialized (cannot access, triggers TDZ error).
- **Function declarations:** Hoisted with complete function body definition safely.

Simple example

javascript

```
console.log(a); // var returns undefined

var a = 10;

sayHello(); // prints output safely since function declarations are fully hoisted

function sayHello() {
   console.log("Hello!");
}
```

Output undefined Hello!

---

2. Temporal Dead Zone (TDZ)

What is it? let aur const variables declare code block start point aur actual initialization lines boundaries ke beech ke access time period limits (dead zone) ko TDZ bolte hain.

Why let/const behave differently? var registers and automatically assigns undefined in memory allocation phase. let/const memory address book sets to uninitialized. Uninitialized variable lookups throws runtime exceptions.

Simple example

javascript

```
// TDZ area active for variable x
// console.log(x); // ReferenceError: Cannot access 'x' before initialization
let x = 99; // TDZ ends here
console.log(x);
```

PART 3: CLOSURES & ADVANCED LOGIC

1. Closures

What is it? Closure inner functions capabilities properties data sets boundaries control memory link parameters settings check coordinates. Inner function returns coordinates outer lexical scopes variable addresses state values lock in memory.

Simple example

javascript

```
function multiplier(x) {
   return function(y) {
      return x * y; // x is saved in lexical scope memory boundary
   };
}
let double = multiplier(2);
console.log(double(10));
```

Output 20

Practical secure wallet simulation:

javascript

```
function secureVault(passcode) {
   let code = passcode;
```

```
    return {
        check: (input) => input === code
    };
}
let vault = secureVault("secret123");
console.log(vault.check("wrong"));     // false
console.log(vault.check("secret123")); // true
```

---

2. Callbacks & Higher-Order Functions

Connect learning details:

- **Callback:** Function passed as arguments input target parameters list.
- **Higher-Order Function:** Function that receives callback or returns function.

Simple example

javascript

```
let list = [1, 2, 3];
let triple = x => x * 3; // callback
console.log(list.map(triple)); // map is HOF
```

Output [3, 6, 9]

---

3. The this Keyword

What is it? this keyword represents execution environment objects contexts.

Basic behavior

- **Object method context:** points to owner parent object context block properties.
- **Global scope context:** points to global window (node environment objects contexts).
- **Arrow function scoping:** arrow functions do not possess local this context binding, they dynamically inherit lexically from outer nesting scope blocks.

Simple example

javascript

```
let profile = {
   username: "Aman",
   printName: function() {
      console.log(this.username);
   }
};
profile.printName();
```

Output Aman

🧠 QUICK REVISION SUMMARY

- **Scopes & chain:** Physical allocation nesting defines Lexical scope. Search traces parent links boundaries.
- **Execution context:** Memory phase mapping → Code line execution blocks. Call stack handles order executions.
- **Hoisting:** var variables defaults undefined; let/const block bounds throws ReferenceError under TDZ; function declarations hoist fully.
- **Closures:** Inner functions retaining parent scope variable values in memory even after outer scopes closed.
- **this keyword:** Points to active execution context owner targets. Arrow functions resolve this lexically.

📑 IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Temporal Dead Zone:** Scope period time limit area constraints where let/const exists in memory uninitialized.
2. **Closure:** Inner function references environment binding that locks variables states memory.
3. **Execution Context:** The workspace environments parsing memory allocations lists and executing lines code in JavaScript.
4. **Scope Chain:** Linked list lookup paths connecting nesting scopes hierarchies up to global.

5. **Static Scoping (Lexical):** The design standard where variables scoping blocks are evaluated static parsing code files lines.

💬 **IMPORTANT INTERVIEW QUESTIONS & ANSWERS**

Q1. Difference between Scope and Scope Chain?

Ans: Scope defines variable visibility boundaries. Scope Chain is the link reference lookup paths linking nesting blocks up to the global scope.

Q2. Explain Closures with a simple technical definition.

Ans: Closure is the reference lock matching inner functions to parent variables scope environments, allowing inner function to remember variable values even after parent scope closes.

Q3. Predict output:

javascript

console.log(val);

var val = 10;

Ans: undefined. Variable val is hoisted and initialized with default undefined.

Q4. Predict output:

javascript

console.log(num);

let num = 20;

Ans: ReferenceError. Early accesses to let variables trigger Temporal Dead Zone constraints.

Q5. What is temporal dead zone (TDZ)?

Ans: The physical area in scope where let/const variables exist in memory but cannot be accessed before their declaration line is executed.

Q6. Difference between var, let, and const in scoping?

Ans: var is function-scoped and hoisted with default undefined. let and const are block-scoped and hoisted without initialization (restricted by TDZ).

Q7. What does the Call Stack track?

Ans: It tracks active execution contexts in a Last In First Out (LIFO) stack order to manage program execution threads.

Q8. Predict output:

javascript

```
let x = 10;
function test() {
   console.log(x);
}
test();
```

Ans: 10. Lexical scope lookup links inner function to the outer variable x.

Q9. What are Higher-Order Functions?

Ans: Functions that take another function as an argument, or return a function as a result.

Q10. Predict output:

javascript

```
const profile = {
   username: "Ravi",
   greet: () => {
      console.log(this.username);
   }
};
profile.greet();
```

Ans: undefined. Arrow functions inherit this lexically. Since profile object does not have its own execution scope, this refers to the global window context where username is undefined.

Q11. Can we call function declarations before writing them in code?

Ans: Yes. Function declarations are hoisted completely with their function body.

Q12. Predict output of function expressions accessed early.

Ans: TypeError (if declared with var, because it is hoisted as undefined) or ReferenceError (if declared with let/const, due to TDZ).

Q13. How do closures prevent memory cleanups of scope variables?

Ans: Outer variables referenced by inner functions are locked in memory as long as the inner closure function exists.

Q14. Predict output:

javascript

```
for (var i = 0; i < 3; i++) {
   setTimeout(() => console.log(i), 100);
}
```

Ans: 3 3 3. var is function-scoped. By the time setTimeout callbacks run, loop execution has completed and the shared variable i is 3.

Q15. Predict output of above loop using let instead of var:

Ans: 0 1 2. let is block-scoped, creating a new variable binding for each loop iteration.

Q16. Predict output:

javascript

```
function parent() {
   let parentVar = 100;
   return () => parentVar;
}
let getVal = parent();
console.log(getVal());
```

Ans: 100. Returns parentVar values via closure.

Q17. What is lexical scoping?

Ans: Scoping mechanism where variable accessibility is determined by their physical location in the nested structure of written source code.

Q18. Predict output:

javascript

```
let x = 10;
function change() {
   x = 20;
}
```

change();

console.log(x);

Ans: 20. The variable is updated globally since it was not re-declared in the local scope.

Q19. Does arrow function bind its own execution context this?

Ans: No. It inherits this context from its enclosing lexical parent scope.

Q20. What is global execution context default creation limit?

Ans: Only 1 Global Execution Context is created per script execution.

💻 PRACTICE QUESTIONS

Question 1: Check Temporal Dead Zone

- **Question:** Write code demonstrating let/const behaviour inside TDZ.
- **Solution:**

javascript

```
function testTDZ() {
  try {
    console.log(x); // accessing inside TDZ
  } catch (err) {
    console.log("Error caught: " + err.message);
  }
  let x = 10;
}
testTDZ();
```

- **Output:** Error caught: Cannot access 'x' before initialization
- **Explanation:** Checks let variable before declaration line, throwing a ReferenceError caught by the try-catch block.

Question 2: Closure counter factory

- **Question:** Create a counter helper function using closures to secure internal counter variables.

- **Solution:**

javascript

```
function createCounter() {
   let val = 0;
   return () => {
      val++;
      return val;
   };
}
let countAction = createCounter();
console.log(countAction()); // 1
console.log(countAction()); // 2
```

- **Explanation:** Internal count state is hidden from outside access but can be read/updated by the returned arrow function closure.

Question 3: Dynamic this scoping context

- **Question:** Call normal object methods displaying this context.
- **Solution:**

javascript

```
const carObj = {
   brand: "Toyota",
   showBrand: function() {
      return this.brand;
   }
};
console.log(carObj.showBrand()); // Toyota
```

- **Explanation:** Since showBrand is invoked as a method of carObj, this binds to the owner object carObj.